ARCHITECTURE REPRESENTATION (BIỂU DIỄN KIẾN TRÚC)

- Biểu diễn kiến trúc phần mềm: Cầu nối truyền đạt cấu trúc, hành vi và các quyết định kỹ thuật của hệ thống.
- Vì sao một sơ đồ duy nhất không bao giờ là đủ?
  - Mỗi nhóm Stakeholders có mối quan tâm riêng (Khách hàng, Lập trình viên, Kiến trúc sư, Vận hành SRE)
  - Hệ thống phần mềm có nhiều chiều: Cấu trúc logic, tổ chức mã nguồn, luồng tiến trình chạy và hạ tầng vật lý
- Phương pháp luận chính: Mô hình 4+1 Views (Kruchten), C4 Model (Simon Brown) và Mô hình Dữ liệu.
- Hệ thống kiểm chứng thực nghiệm: Google Online Boutique & Mini Shop Microservices.

PHÂN BIỆT RÕ RÀNG: VIEW VS. NOTATION

Architecture View (Góc nhìn)

- Là góc nhìn biểu diễn hệ thống nhằm giải quyết mối quan tâm (Concerns) của nhóm Stakeholder cụ thể
- 4 sơ đồ UML đơn lẻ KHÔNG đồng nghĩa với 4 Views kiến trúc nếu thiếu ranh giới, ngữ cảnh và trạng thái
- Ví dụ: Dev quan tâm giao diện hợp đồng API, SRE quan tâm Pod/Container, Port và tài nguyên CPU/RAM

Notation (Ký hiệu vẽ)

- Là ngôn ngữ hình học dùng để vẽ (Boxes & Arrows, UML Class/Sequence, C4, BPMN)
- Nguyên tắc bắt buộc: Phải có chú giải (Legend), nhãn giao thức (HTTP, gRPC) và chiều mũi tên
- Một sơ đồ kiến trúc hoàn chỉnh luôn phải có văn bản diễn giải trách nhiệm và ràng buộc dữ liệu đi kèm

Mô hình kiến trúc · Khung trục 4+1

KHUNG TRỤC MÔ HÌNH 4+1 VIEW MODEL

1.Logical View

- Phân rã chức năng
- Ranh giới dịch vụ
- Quyền sở hữu dữ liệu
- Dịch vụ thật vs. Mock

2.Development View

- Tổ chức thư mục/mã
- Quản lý package nội bộ
- Hợp đồng giao tiếp chung
- Chiến lược build Docker

3.Process View

- Luồng chạy runtime động
- Chuỗi đồng bộ vs. sự kiện
- Xử lý Timeout & Retry
- Giao dịch dữ liệu (ACID)

4.Physical / Deploy

- Ánh xạ vào máy/server
- Container, Pod & Service
- Mạng mạng nội bộ vs. biên
- Cô lập lỗi khi sập node

LOGICAL VIEW & DEVELOPMENT VIEW (TỔ CHỨC SOURCE)

- Logical View: Phân định ranh giới trách nhiệm và sở hữu dữ liệu độc lập:
  - API Gateway / BFF (Frontend): Điểm tiếp nhận request từ Client, giữ session và chuyển đổi giao thức
  - Orchestrator (Checkout/Orders): Điều phối luồng nghiệp vụ giữa các microservices độc lập
  - Phân loại Dịch vụ Thật (ghi nhận dữ liệu) vs. Dịch vụ Mock (mô phỏng thanh toán, vận chuyển, email)
- Development View: Tổ chức mã nguồn đảm bảo tính độc lập và khả năng triển khai:
  - Nguyên tắc Share-Nothing lúc Runtime: Các dịch vụ không phụ thuộc thư viện dùng chung dễ gây gãy đổ chéo
  - Đóng gói độc lập qua Multi-stage Dockerfile: Mỗi dịch vụ có thể sửa đổi, build lại mà không cần restart toàn hệ thống
  - Định nghĩa hợp đồng giao tiếp rõ ràng: Chuẩn hóa bằng Protobuf (demo.proto) hoặc OpenAPI/JSON Schema

PROCESS VIEW: ĐIỀU PHỐI ĐỒNG BỘ, TIMEOUT & GIAO DỊCH

- Process View thể hiện hành vi tương tác động giữa các thành phần lúc hệ thống vận hành:
  - Luồng điều phối tuần tự (Synchronous Orchestration): Checkoutservice hoặc Orders Service gọi chuỗi dịch vụ con
  - Quy luật bậc thang Timeout: Giảm dần từ ngoài vào trong (Nginx 10s -> Gateway 8s -> Service nội bộ 3s) tránh treo luồng
- Quản lý tính toàn vẹn và chống trùng lặp (Idempotency):
  - Header Idempotency-Key & mã băm SHA-256 payload: Chống tạo 2 hóa đơn trùng lặp khi mạng chập chờn
  - Giao dịch dữ liệu (ACID Transaction): Ghi nguyên tử trên từng cơ sở dữ liệu độc lập, tự động rollback khi có lỗi

Mô hình kiến trúc · C4 Model (Simon Brown)

C4 MODEL: 4 CẤP ĐỘ PHÓNG ĐẠI KIẾN TRÚC TĨNH

C1: Context

- Bức tranh toàn cảnh
- Người dùng (Actors)
- Hệ thống bên ngoài
- Ranh giới hệ thống

C2: Container

- Ứng dụng deployable
- Frontend, API, DB
- Giao thức kết nối
- Đơn vị triển khai

C3: Component

- Cấu trúc trong 1 app
- Controllers, Handlers
- Service, Repository
- Ranh giới nội bộ

C4: Code

- Chi tiết hàm / class
- Đối tượng Domain
- Đoạn mã cốt lõi
- Ánh xạ file:line

Thiết kế Dữ liệu · Data Model / Database Schema

BIỂU DIỄN DỮ LIỆU: QUAN HỆ VS. PHI QUAN HỆ

Database-per-Service (Mini Shop)

- Mỗi dịch vụ sở hữu 1 cơ sở dữ liệu riêng (products-db và orders-db riêng biệt)
- Tuyệt đối không dùng Foreign Key vật lý nối xuyên qua các cơ sở dữ liệu độc lập
- Cơ chế Snapshot Giá: Sao chép cứng tên và giá vào đơn hàng tại thời điểm mua (không đổi khi Admin sửa giá)

In-Memory & Protobuf (Online Boutique)

- Kiến trúc Cloud-Native siêu nhẹ: Lưu giỏ hàng tạm bằng Redis (Key = user_id, Value = byte Protobuf)
- Dữ liệu danh mục nạp trực tiếp vào RAM (products.json) tối ưu I/O
- Mô hình tiền tệ chuẩn xác (Exact Money Model): Tách units (nguyên) và nanos (lẻ) tránh sai số IEEE 754

BẢO MẬT CONTAINER, XỬ LÝ ĐỒNG THỜI & +1 SCENARIOS

- Security View (Biểu diễn vùng tin cậy & bảo mật hạ tầng):
  - Xác lập 3 vùng biên tin cậy (Trust Boundaries): Trình duyệt Internet -> Cổng API Biên -> Mạng nội bộ gRPC/SQL
  - Pod SecurityContext / Non-Root: Chạy người dùng giới hạn (UID 1000), bật readOnlyRootFilesystem chống can thiệp tệp
- Concurrency View (Biểu diễn xử lý đồng thời & Race Condition):
  - Rủi ro Lost Update trong mô hình Read-Modify-Write: Cùng 1 user bắn 2 request đồng thời nếu không có khóa
  - Chặn race condition bằng ràng buộc cơ sở dữ liệu: Khóa UNIQUE(idempotency_key) hoặc xử lý luồng Async
- +1 Scenarios: Sử dụng các kịch bản thực tế (mua hàng, gián đoạn mạng, sập node) để kiểm chứng lại toàn bộ 4 views!

ÁNH XẠ KIẾN TRÚC: HỘP SƠ ĐỒ -> MÃ NGUỒN -> RUNTIME

- Một bản biểu diễn kiến trúc chỉ có giá trị khi kiểm chứng được trực tiếp trên hệ thống thực tế.
- Quy tắc ánh xạ 10 trường theo chuẩn phân công kiến trúc:
  - Tên hộp trên sơ đồ (Component Name) & Trách nhiệm nghiệp vụ (Single Responsibility)
  - Giao thức & Hợp đồng giao tiếp (Interface Contract: REST endpoint, gRPC service trong demo.proto)
  - Vị trí trong mã nguồn (Source symbol, file:line cụ thể trong src/)
  - Thành phần chạy thực tế (Runtime Component: Docker Container, K8s Pod, Service IP, Cổng lắng nghe)
  - Bằng chứng vận hành (Runtime Evidence: Trạng thái Running, Log giao dịch và kiểm tra tính chịu lỗi)

BẰNG CHỨNG RUNTIME: LOG ĐẶT HÀNG & GIAO DỊCH THẬT

PowerShell — Kubernetes Runtime Logs: Luồng Mua Sản Phẩm Online Boutique (theanh-architecture)

```
PS> curl.exe -i -X POST http://localhost:8088/cart/checkout -H 'Cookie: shop_session-id=theanh-session-01'
HTTP/1.1 200 OK   |   Date: Tue, 03 Nov 2026 14:53:49 GMT   |   Content-Type: text/html; charset=utf-8
  <div class='col-6'>Tracking #</div>   <div class='col-6'>Total Paid: USD 48.97</div>
-------------------------------------------------------------------------------------------------------
PS> kubectl logs -n theanh-architecture deploy/checkoutservice --tail=3
{"message":"[PlaceOrder] user_id=\"theanh-session-01\" user_currency=\"USD\"","severity":"info","timestamp":"2026-11-03T14:53:49.918Z"}
{"message":"payment went through (transaction_id: 29c530dc-99eb-42b7-a0c1-999daa0dc6df)","severity":"info"}
{"message":"order confirmation email sent to \"theanh@example.com\"","severity":"info"}
-------------------------------------------------------------------------------------------------------
PS> kubectl logs -n theanh-architecture deploy/paymentservice --tail=2
{"name":"paymentservice-server","message":"PaymentService#Charge invoked: {\"currency_code\":\"USD\",\"units\":\"48\",\"nanos\":970000000}"}
{"name":"paymentservice-charge","message":"Transaction processed: visa ending 0707    Amount: USD48.970000000"}
-------------------------------------------------------------------------------------------------------
PS> kubectl logs -n theanh-architecture deploy/emailservice --tail=1
{"severity":"INFO","name":"emailservice-server","message":"Order confirmation email to theanh@example.com received."}
PS> kubectl exec -n theanh-architecture deploy/redis-cart -- redis-cli GET 'theanh-session-01'
(nil)  -> Giỏ hàng Redis đã được xóa sạch sau bước EmptyCart(:7070) của PlaceOrder()!
```

Kiểm chứng Process View: Request POST checkout đi qua Gateway -> Orchestrator gọi thanh toán, gửi email xác nhận và làm rỗng giỏ hàng (Áp dụng đồng nhất trên cả K8s Online Boutique và Docker Mini Shop).

BẰNG CHỨNG RUNTIME: TẮT DỊCH VỤ & CÔ LẬP LỖI (200 VS 500)

PowerShell — Kiểm Chứng Cô Lập Lỗi Online Boutique: Tắt adservice (200 OK) vs. Tắt redis-cart (HTTP 500)

```
PS> # 1. TẮT DỊCH VỤ PHỤ TRỢ (NON-CRITICAL): adservice -> Trang web vẫn sống 200 OK
PS> kubectl scale deploy/adservice -n theanh-architecture --replicas=0
deployment.apps/adservice scaled (0/1 Ready) -> chooseAd() timeout 100ms tại src/frontend/rpc.go:169
PS> curl.exe -I http://localhost:8088/product/OLJCESPC7Z
HTTP/1.1 200 OK   |   Set-Cookie: shop_session-id=454b10c4-6f4c-401c... (Chỉ ghi log.Warn, không sập trang)
-------------------------------------------------------------------------------------------------------
PS> # 2. TẮT DỊCH VỤ CỐT LÕI (CRITICAL): redis-cart -> Báo lỗi ngay HTTP 500 Internal Server Error
PS> kubectl scale deploy/redis-cart -n theanh-architecture --replicas=0
PS> curl.exe -i http://localhost:8088/cart -H 'Cookie: shop_session-id=theanh-session-01'
HTTP/1.1 500 Internal Server Error   |   Date: Tue, 03 Nov 2026 14:53:59 GMT
Error: could not retrieve cart: rpc error: code = FailedPrecondition desc = Can't access cart storage.
  -> Ném từ RedisCartStore.cs:102 -> frontend/handlers.go:219 gọi renderHTTPError(w, r, err, 500)
-------------------------------------------------------------------------------------------------------
PS> # 3. KHÔI PHỤC DỊCH VỤ: Bật lại redis-cart --replicas=1 -> Tự động phục hồi HTTP 200 OK
PS> kubectl scale deploy/redis-cart -n theanh-architecture --replicas=1
deployment.apps/redis-cart condition met   ->   HTTP/1.1 200 OK (Khôi phục hoàn toàn không cần restart frontend)
```

Kiểm chứng +1 Scenarios & Deployment View: Tắt dịch vụ phụ trợ (adservice) hệ thống vẫn 200 OK; tắt dịch vụ lưu trữ cốt lõi (redis-cart / products) hệ thống trả lỗi có kiểm soát (500/503) và tự phục hồi khi bật lại.

ĐÁNH ĐỔI KIẾN TRÚC (ARCHITECTURAL TRADE-OFFS)

- Mọi quyết định biểu diễn và thiết kế kiến trúc đều là sự đánh đổi có chủ đích (Trade-offs):
  - Database-per-Service vs. Tính nhất quán: Đổi lấy cô lập lỗi tuyệt đối nhưng chấp nhận dữ liệu lặp lại ở bảng snapshot
  - Polyglot gRPC vs. Chatty Calls: Giao tiếp chặt chẽ đa ngôn ngữ trên HTTP/2 nhưng cần BFF chuyển đổi giao thức
  - Sync Orchestrator vs. Event-Driven: Luồng chạy tập trung dễ trace/debug mã nguồn nhưng tạo điểm nghẽn phụ thuộc
- Kết luận về Biểu diễn Kiến trúc Phần mềm:
  - Biểu diễn kiến trúc không phải là vẽ tranh, mà là làm rõ ranh giới, hợp đồng và hành vi của hệ thống
  - Sự kết hợp giữa 4+1 Views + C4 Model + Bằng chứng Runtime tạo nên bức tranh toàn diện và chính xác nhất